

Experiment 1 Installation and Configuration of STM32CubeIDE and Creation of the First STM32 Project

```
/* Auto-generated skeleton from STM32CubeIDE / CubeMX (excerpt) */
int main(void)
{
    HAL_Init();           /* Reset peripherals, init Flash & SysTick */
    SystemClock_Config(); /* Configure the system clock (180 MHz) */
    MX_GPIO_Init();       /* Initialize all configured GPIO ports */

    while (1)
    {
        /* Application code goes here */
    }
}
```

Experiment 2 Implementing and Analyzing GPIO Programming: Blinking the Onboard LED Using STM32 HAL

```
while (1)
{
    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5); /* Toggle LD2 */
    HAL_Delay(500);                         /* Blocking 500 ms delay */
}
```

Experiment 3 Development of a SysTick Timer-Based Periodic LED Task Scheduler Without Using Software Delays

```
uint32_t lastToggleTick = 0;
const uint32_t LED_PERIOD_MS = 500;

while (1)
{
    uint32_t now = HAL_GetTick();
    if ((now - lastToggleTick) >= LED_PERIOD_MS)
    {
        lastToggleTick = now;
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
    }
    /* Additional non-blocking tasks can be polled here */
}
```

Experiment 4 Constructing a Debounced GPIO Input Interface: Push Button Controlled LED Operation

```
#define DEBOUNCE_MS 30

GPIO_PinState lastRead = GPIO_PIN_SET;
GPIO_PinState acceptedState = GPIO_PIN_SET;
uint32_t lastDebounceTick = 0;

while (1)
{
    GPIO_PinState reading = HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13);

    if (reading != lastRead)
    {
        lastDebounceTick = HAL_GetTick();
        lastRead = reading;
    }

    if ((HAL_GetTick() - lastDebounceTick) > DEBOUNCE_MS)
    {
        if (reading != acceptedState)
        {
            acceptedState = reading;
            if (acceptedState == GPIO_PIN_RESET) /* Active-low press */
            {
                HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
            }
        }
    }
}
```

Experiment 5 Evaluating GPIO Drive Strengths: Performance Analysis of GPIO Speed and Pull-Up/Pull-Down Configurations

```
GPIO_InitTypeDef GPIO_InitStructure = {0};

/* --- Output speed comparison on PA6 --- */
GPIO_InitStructure.Pin = GPIO_PIN_6;
GPIO_InitStructure.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStructure.Pull = GPIO_NOPULL;
GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_VERY_HIGH; /* try _LOW, _MEDIUM, _HIGH too */
HAL_GPIO_Init(GPIOA, &GPIO_InitStructure);

/* --- Floating-input pull test on PB0 --- */
GPIO_InitStructure.Pin = GPIO_PIN_0;
GPIO_InitStructure.Mode = GPIO_MODE_INPUT;
GPIO_InitStructure.Pull = GPIO_PULLUP; /* try GPIO_NOPULL, GPIO_PULLDOWN */
HAL_GPIO_Init(GPIOB, &GPIO_InitStructure);
```

Experiment 6 Implementing Logic-Level Control for High-Power Relay Interfacing and Digital Output Control Using STM32

```
#define RELAY_PIN    GPIO_PIN_0
#define RELAY_PORT   GPIOB
/* Most low-cost relay modules are ACTIVE-LOW: driving IN low energizes the coil */

void Relay_Set(uint8_t energize)
{
    HAL_GPIO_WritePin(RELAY_PORT, RELAY_PIN,
                      energize ? GPIO_PIN_RESET : GPIO_PIN_SET);
}

while (1)
{
    Relay_Set(1);      /* Energize: relay clicks, NO contact closes */
    HAL_Delay(2000);
    Relay_Set(0);      /* De-energize */
    HAL_Delay(2000);
}
```

Experiment 7 Interfacing an I²C 16×2 LCD and Displaying User Information with Relay Status

```
/* Using a minimal PCF8574 LCD driver (I2C address 0x27 << 1 = 0x4E) */
#define LCD_I2C_ADDR (0x27 << 1)

Lcd_Init(&hi2c1, LCD_I2C_ADDR);
Lcd_SetCursor(0, 0);
Lcd_Print("Roll No: 21EC045");

while (1)
{
    Lcd_SetCursor(1, 0);
    if (HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_0) == GPIO_PIN_RESET) /* active-low = energized */
        Lcd_Print("Relay: ON ");
    else
        Lcd_Print("Relay: OFF");

    HAL_Delay(200);
}
```

Experiment 8 Developing and Analyzing UART Serial Communication: Data Transmission from STM32 to PC

```
char msg[64];
uint32_t counter = 0;

HAL_UART_Transmit(&huart2, (uint8_t*)"STM32 UART TX Demo\r\n", 20, HAL_MAX_DELAY);

while (1)
{
    int len = sprintf(msg, "Counter = %lu\r\n", counter++);
    HAL_UART_Transmit(&huart2, (uint8_t*)msg, len, HAL_MAX_DELAY);
    HAL_Delay(1000);
}
```

Experiment 9 Developing and Analyzing UART Serial Communication: Data Reception from PC to STM32

```
uint8_t rxByte;

while (1)
{
    HAL_UART_Receive(&huart2, &rxByte, 1, HAL_MAX_DELAY);

    if (rxByte == '1')
    {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
        HAL_UART_Transmit(&huart2, (uint8_t*)"LED ON\r\n", 8, HAL_MAX_DELAY);
    }
    else if (rxByte == '0')
    {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
        HAL_UART_Transmit(&huart2, (uint8_t*)"LED OFF\r\n", 9, HAL_MAX_DELAY);
    }
    else
    {
        HAL_UART_Transmit(&huart2, (uint8_t*)"Unknown command\r\n", 18, HAL_MAX_DELAY);
    }
}
```

Experiment 10 Development of a UART Echo Application Using STM32 Displaying on LCD

```
char line[17];
uint8_t idx = 0;
uint8_t rxByte;

Lcd_Init(&hi2c1, LCD_I2C_ADDR);
Lcd_SetCursor(0, 0);
Lcd_Print("UART->LCD Echo");

while (1)
{
    HAL_UART_Receive(&huart2, &rxByte, 1, HAL_MAX_DELAY);
    HAL_UART_Transmit(&huart2, &rxByte, 1, HAL_MAX_DELAY);    /* echo */

    if (rxByte == '\r')
    {
        line[idx] = '\0';
        Lcd_SetCursor(1, 0);
        Lcd_Print("                ");    /* clear row */
        Lcd_SetCursor(1, 0);
        Lcd_Print(line);
        idx = 0;
    }
    else if (idx < 16)
    {
        line[idx++] = rxByte;
    }
}
```

Experiment 11 Designing Asynchronous Data Interrupt-Driven UART Communication Using HAL Library

```
uint8_t rxByte;
volatile uint32_t idleLoops = 0;  /* proves the main loop is not blocked */

int main(void)
{
    /* ... HAL_Init, SystemClock_Config, MX_GPIO_Init, MX_USART2_UART_Init ... */

    HAL_UART_Receive_IT(&huart2, &rxByte, 1);  /* arm first interrupt-driven RX */

    while (1)
    {
        idleLoops++;  /* main loop keeps running, unblocked, between characters */
    }
}

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART2)
    {
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
        HAL_UART_Receive_IT(&huart2, &rxByte, 1);  /* re-arm for the next byte */
    }
}
```

Experiment 12 Synthesizing Data Streams from UART to I2C Peripheral Displays

```
volatile uint8_t lineReady = 0;
char lineBuf[32];
uint8_t lineIdx = 0;
uint8_t rxByte;

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART2)
    {
        if (rxByte == '\n' || lineIdx >= sizeof(lineBuf) - 1)
        {
            lineBuf[lineIdx] = '\0';
            lineReady = 1;
            lineIdx = 0;
        }
        else
        {
            lineBuf[lineIdx++] = rxByte;
        }
        HAL_UART_Receive_IT(&huart2, &rxByte, 1);
    }
}

/* In the main loop: */
if (lineReady)
{
    if (strncmp(lineBuf, "L1:", 3) == 0) { Lcd_SetCursor(0,0); Lcd_Print(&lineBuf[3]); }
    if (strncmp(lineBuf, "L2:", 3) == 0) { Lcd_SetCursor(1,0); Lcd_Print(&lineBuf[3]); }
    lineReady = 0;
}
```

Experiment 13 Developing a Rule-Based Automated Control System Using UART Communication

```
typedef enum { MODE_AUTO, MODE_MANUAL } ControlMode;
ControlMode mode = MODE_AUTO;
int threshold = 50;
int level = 0;          /* e.g. populated from an ADC reading */

void ApplyRules(void)
{
    if (mode == MODE_AUTO)
    {
        Relay_Set(level > threshold);  /* IF level exceeds threshold THEN energize */
    }
    /* In MODE_MANUAL, relay state is instead set directly by RELAY:ON / RELAY:OFF commands */
}

/* Called from the main loop once a full command line has been parsed: */
void HandleCommand(char *cmd)
{
    if (strncmp(cmd, "MODE:AUTO", 9) == 0) mode = MODE_AUTO;
    else if (strncmp(cmd, "MODE:MANUAL", 11) == 0) mode = MODE_MANUAL;
    else if (strncmp(cmd, "SET:", 4) == 0) threshold = atoi(&cmd[4]);
    else if (strncmp(cmd, "RELAY:ON", 8) == 0 && mode == MODE_MANUAL) Relay_Set(1);
    else if (strncmp(cmd, "RELAY:OFF", 9) == 0 && mode == MODE_MANUAL) Relay_Set(0);
}
```

Experiment 14 Designing a PWM Signal Generation Using STM32 Timers

```
/* TIM3 configured: PSC = 89, ARR = 999 -> 90 MHz / (90 x 1000) = 1 kHz PWM */

HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);

uint32_t duty = 0;
int8_t step = 1;

while (1)
{
    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, duty); /* duty in [0, ARR] */

    duty += step;
    if (duty >= 999 || duty == 0) step = -step; /* ramp up/down */

    HAL_Delay(5);
}
```

Experiment 15 Designing a Servo Motor Position Control Using PWM

```
/* TIM3: PSC = 89, ARR = 19999 -> 1 us/tick, 20 ms (50 Hz) frame */
#define SERVO_MIN_US 1000 /* 0 degrees */
#define SERVO_MID_US 1500 /* 90 degrees */
#define SERVO_MAX_US 2000 /* 180 degrees */

HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);

__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, SERVO_MIN_US);
HAL_Delay(1000);
__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, SERVO_MID_US);
HAL_Delay(1000);
__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, SERVO_MAX_US);
HAL_Delay(1000);
```

Experiment 16 Design and Implementation of a Bidirectional PWM-Controlled DC Motor Drive Using an H-Bridge Driver

```
#define IN1_PIN GPIO_PIN_1
#define IN2_PIN GPIO_PIN_2
#define MOTOR_PORT GPIOB

void Motor_Forward(void) {
    HAL_GPIO_WritePin(MOTOR_PORT, IN1_PIN, GPIO_PIN_SET);
    HAL_GPIO_WritePin(MOTOR_PORT, IN2_PIN, GPIO_PIN_RESET);
}
void Motor_Reverse(void) {
    HAL_GPIO_WritePin(MOTOR_PORT, IN1_PIN, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(MOTOR_PORT, IN2_PIN, GPIO_PIN_SET);
}
void Motor_Stop(void) {
    HAL_GPIO_WritePin(MOTOR_PORT, IN1_PIN, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(MOTOR_PORT, IN2_PIN, GPIO_PIN_RESET);
}
void Motor_SetSpeed(uint32_t duty) { /* duty in [0, ARR] */
    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, duty);
}

HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);
Motor_SetSpeed(500); /* ~50% of ARR = 999 */
Motor_Forward(); HAL_Delay(3000);
Motor_Stop(); HAL_Delay(1000);
Motor_Reverse(); HAL_Delay(3000);
Motor_Stop();
```

Experiment 17 Designing a UART-Based Command-Controlled Servo Motor Positioning

```
int ClampAngle(int angle)
{
    if (angle < 0)    return 0;
    if (angle > 180) return 180;
    return angle;
}

/* Called once a full line (terminated by '\r') has been received over UART: */
void HandleServoCommand(char *line)
{
    char *endptr;
    long angle = strtol(line, &endptr, 10);

    if (endptr == line)                /* not a valid number */
    {
        HAL_UART_Transmit(&huart2, (uint8_t*)"Invalid input\r\n", 15, HAL_MAX_DELAY);
        return;
    }

    angle = ClampAngle((int)angle);
    uint32_t pulse_us = 1000 + (angle * 1000) / 180;    /* 1000-2000 us range */
    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, pulse_us);

    char msg[32];
    int len = sprintf(msg, "Moved to %ld deg\r\n", angle);
    HAL_UART_Transmit(&huart2, (uint8_t*)msg, len, HAL_MAX_DELAY);
}
```

Experiment 18 Designing a Single-Channel ADC Interfacing for Potentiometer Voltage Measurement

```
HAL_ADC_Start(&hadc1);

while (1)
{
    HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY);
    uint32_t raw = HAL_ADC_GetValue(&hadc1);
    float volts = (raw / 4095.0f) * 3.3f;

    char msg[48];
    int len = sprintf(msg, "Raw=%lu Voltage=%.2fV\r\n", raw, volts);
    HAL_UART_Transmit(&huart2, (uint8_t*)msg, len, HAL_MAX_DELAY);

    HAL_Delay(500);
}
```

Experiment 19 Designing a Multi-Channel ADC Interfacing for LM35 Temperature Sensor Measurement

```
HAL_ADC_Start(&hadc1);

while (1)
{
    HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY);
    uint32_t raw = HAL_ADC_GetValue(&hadc1);

    float voltage_mV = (raw / 4095.0f) * 3300.0f;    /* VREF = 3.3 V = 3300 mV */
    float tempC = voltage_mV / 10.0f;              /* LM35: 10 mV per degC */

    char msg[48];
    int len = sprintf(msg, "Raw=%1u Temp=%.1f C\r\n", raw, tempC);
    HAL_UART_Transmit(&huart2, (uint8_t*)msg, len, HAL_MAX_DELAY);

    HAL_Delay(1000);
}
```

Experiment 20 Multi-Channel ADC Interfacing for LDR-Based Ambient Light Measurement

```
HAL_ADC_Start(&hadc1);

while (1)
{
    HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY);
    uint32_t raw = HAL_ADC_GetValue(&hadc1);
    float lightPct = (raw / 4095.0f) * 100.0f;

    char msg[48];
    int len = sprintf(msg, "Raw=%lu  Light=%.1f%%\r\n", raw, lightPct);
    HAL_UART_Transmit(&huart2, (uint8_t*)msg, len, HAL_MAX_DELAY);

    HAL_Delay(500);
}
```

Experiment 21 Displaying LM35 Temperature and LDR Light Intensity on an I²C LCD

```
uint32_t ReadChannel(uint32_t channel)
{
    ADC_ChannelConfTypeDef sConfig = {0};
    sConfig.Channel = channel;
    sConfig.Rank = ADC_REGULAR_RANK_1;
    sConfig.SamplingTime = ADC_SAMPLETIME_56CYCLES;
    HAL_ADC_ConfigChannel(&hadc1, &sConfig);

    HAL_ADC_Start(&hadc1);
    HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY);
    return HAL_ADC_GetValue(&hadc1);
}

while (1)
{
    float tempC      = (ReadChannel(ADC_CHANNEL_1) / 4095.0f) * 3300.0f / 10.0f;
    float lightPct = (ReadChannel(ADC_CHANNEL_4) / 4095.0f) * 100.0f;

    char line0[17], line1[17];
    sprintf(line0, "Temp: %5.1f C", tempC);
    sprintf(line1, "Light: %4.1f %%", lightPct);

    Lcd_SetCursor(0, 0); Lcd_Print(line0);
    Lcd_SetCursor(1, 0); Lcd_Print(line1);

    HAL_Delay(500);
}
```

Experiment 22 Intelligent Environmental Control: Relay Switching Based on LDR and DC Motor Speed Control Based on LM35

```
#define LIGHT_THRESHOLD_PCT 30.0f
#define TEMP_MIN_C 25.0f
#define TEMP_MAX_C 35.0f

while (1)
{
    float tempC      = (ReadChannel(ADC_CHANNEL_1) / 4095.0f) * 3300.0f / 10.0f;
    float lightPct = (ReadChannel(ADC_CHANNEL_4) / 4095.0f) * 100.0f;

    /* Rule 1: automatic lamp */
    Relay_Set(lightPct < LIGHT_THRESHOLD_PCT);

    /* Rule 2: proportional fan speed */
    float clampedT = tempC;
    if (clampedT < TEMP_MIN_C) clampedT = TEMP_MIN_C;
    if (clampedT > TEMP_MAX_C) clampedT = TEMP_MAX_C;
    uint32_t duty = (uint32_t)(((clampedT - TEMP_MIN_C) / (TEMP_MAX_C - TEMP_MIN_C)) * 999.0f);

    Motor_Forward();
    Motor_SetSpeed(duty);

    HAL_Delay(500);
}
```

Experiment 23 Monitoring and Transmission of Sensor Data (LM35 & LDR) to PC with Actuator Status Display

```
HAL_UART_Transmit(&huart2, (uint8_t*)"TEMP,LIGHT,RELAY,FAN_DUTY\r\n", 26, HAL_MAX_DELAY);

while (1)
{
    float tempC    = (ReadChannel(ADC_CHANNEL_1) / 4095.0f) * 3300.0f / 10.0f;
    float lightPct = (ReadChannel(ADC_CHANNEL_4) / 4095.0f) * 100.0f;

    uint8_t relayOn = (lightPct < LIGHT_THRESHOLD_PCT);
    Relay_Set(relayOn);

    float clampedT = tempC < TEMP_MIN_C ? TEMP_MIN_C : (tempC > TEMP_MAX_C ? TEMP_MAX_C : tempC);
    uint32_t duty = (uint32_t)(((clampedT - TEMP_MIN_C) / (TEMP_MAX_C - TEMP_MIN_C)) * 999.0f);
    Motor_Forward();
    Motor_SetSpeed(duty);

    char line[64];
    int len = sprintf(line, "%.1f,%.1f,%s,%lu\r\n",
                       tempC, lightPct, relayOn ? "ON" : "OFF", duty);
    HAL_UART_Transmit(&huart2, (uint8_t*)line, len, HAL_MAX_DELAY);

    HAL_Delay(500);
}
```

Experiment 24 Developing a CAN Peripheral Initialization and Loopback Communication Test

```
CAN_TxHeaderTypeDef txHeader = {0};
CAN_RxHeaderTypeDef rxHeader;
uint8_t txData[2] = {0xAB, 0xCD};
uint8_t rxData[8];
uint32_t txMailbox;

txHeader.StdId = 0x100;
txHeader.IDE   = CAN_ID_STD;
txHeader.RTR   = CAN_RTR_DATA;
txHeader.DLC   = 2;

HAL_CAN_Start(&hcan1);
HAL_CAN_AddTxMessage(&hcan1, &txHeader, txData, &txMailbox);

while (HAL_CAN_GetRxFifoFillLevel(&hcan1, CAN_RX_FIFO0) == 0) { /* wait for loopback */ }

HAL_CAN_GetRxMessage(&hcan1, CAN_RX_FIFO0, &rxHeader, rxData);

if (rxHeader.StdId == 0x100 && rxData[0] == 0xAB && rxData[1] == 0xCD)
{
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);    /* LD2 ON = success */
}
```

Experiment 25 Developing a CAN Communication Between Two STM32 Nucleo Boards

```
/* --- Board A: transmitter --- */
CAN_TxHeaderTypeDef txHeader = {.StdId = 0x200, .IDE = CAN_ID_STD, .RTR = CAN_RTR_DATA, .DLC = 1};
uint8_t counter = 0;
uint32_t txMailbox;

HAL_CAN_Start(&hcan1);
while (1)
{
    uint8_t data[1] = { counter++ };
    HAL_CAN_AddTxMessage(&hcan1, &txHeader, data, &txMailbox);
    HAL_Delay(200);
}

/* --- Board B: receiver --- */
CAN_RxHeaderTypeDef rxHeader;
uint8_t rxData[8];

HAL_CAN_Start(&hcan1);
while (1)
{
    if (HAL_CAN_GetRx FifoFillLevel(&hcan1, CAN_RX_FIFO0) > 0)
    {
        HAL_CAN_GetRxMessage(&hcan1, CAN_RX_FIFO0, &rxHeader, rxData);
        char msg[32];
        int len = sprintf(msg, "ID=0x%lX Data=%u\r\n", rxHeader.StdId, rxData[0]);
        HAL_UART_Transmit(&huart2, (uint8_t*)msg, len, HAL_MAX_DELAY);
    }
}
```

Experiment 26 Designing a Sensor Data Acquisition and Transmission Over CAN Bus

```
/* --- Node A: sensor acquisition + CAN transmit --- */
CAN_TxHeaderTypeDef txHeader = {.StdId = 0x300, .IDE = CAN_ID_STD, .RTR = CAN_RTR_DATA, .DLC = 4};
uint32_t txMailbox;

while (1)
{
    float tempC    = (ReadChannel(ADC_CHANNEL_1) / 4095.0f) * 3300.0f / 10.0f;
    float lightPct = (ReadChannel(ADC_CHANNEL_4) / 4095.0f) * 100.0f;

    int16_t tempFixed = (int16_t)(tempC * 10.0f);    /* e.g. 27.4 C -> 274 */
    int16_t lightFixed = (int16_t)(lightPct * 10.0f);

    uint8_t data[4] = { tempFixed & 0xFF, (tempFixed >> 8) & 0xFF,
                        lightFixed & 0xFF, (lightFixed >> 8) & 0xFF };
    HAL_CAN_AddTxMessage(&hcan1, &txHeader, data, &txMailbox);
    HAL_Delay(1000);
}

/* --- Node B: CAN receive + LCD display --- */
if (HAL_CAN_GetRxFifoFillLevel(&hcan1, CAN_RX_FIFO0) > 0)
{
    CAN_RxHeaderTypeDef rxHeader;
    uint8_t rxData[8];
    HAL_CAN_GetRxMessage(&hcan1, CAN_RX_FIFO0, &rxHeader, rxData);

    if (rxHeader.StdId == 0x300)
    {
        int16_t tempFixed = (int16_t)(rxData[0] | (rxData[1] << 8));
        int16_t lightFixed = (int16_t)(rxData[2] | (rxData[3] << 8));

        char line0[17], line1[17];
        sprintf(line0, "Temp: %5.1f C", tempFixed / 10.0f);
        sprintf(line1, "Light: %4.1f %%", lightFixed / 10.0f);
        Lcd_SetCursor(0, 0); Lcd_Print(line0);
        Lcd_SetCursor(1, 0); Lcd_Print(line1);
    }
}
```